

Step 1 — FastAPI Route — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic**, **Intermediate**, **Advanced**.

- [Hint 1 — The route, and typed request/response](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The route, and typed request/response

Basic Version

A FastAPI route is just a Python function with a decorator on it: `@app.post("/run-task")`. Inside it, call Project 4's graph exactly the way `main.py` already does, and return whatever comes back — FastAPI turns a returned dict (or Pydantic model) into a JSON response automatically.

Nothing gets saved anywhere yet. This step is only about proving the graph is reachable over HTTP at all — everything else (database, Docker, rate limiting) is a later step, on purpose.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The README asks for typed Pydantic request/response models, not a bare dict — a `TaskRequest` (with a `topic: str` field) and a `TaskResponse` (with the final report and whatever else is useful, like `approved: bool`). Declaring these as your route's parameter and `response_model` is what gives you FastAPI's automatic `422 Unprocessable Entity` on a bad request for free — send a request missing `topic`, or with the wrong type, and FastAPI validates and rejects it before your function body ever runs, no code from you required.

Split the app into `api/main.py` (creates the `FastAPI()` app, includes the router), `api/routes.py` (the actual route functions), and `api/schemas.py` (the Pydantic models) — this is the standard FastAPI project shape, and keeps the route logic separate from app setup, which matters once there are more routes (`GET /runs/{id}` arrives in Step 2).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Two real problems show up the moment more than one person can call this route at once — and this project's own problems table names the first one directly: "An internal error shows a raw error trace to an API client."

Unsafe error exposure: if `graph.invoke(...)` raises anything — a bad API key, a rate limit from OpenAI, a bug in one of the 5 agents — FastAPI's default behavior is a `500` response with your actual Python traceback in the body. That's a real information leak (file paths, internal variable names, sometimes even parts of your prompt) handed straight to whoever called the API. The fix: catch broadly at the route level, log the full detail server-side (you'll want it for debugging), and return a plain, generic message with the right `5xx` status code to the client.

A blocking call on a single-threaded event loop: FastAPI's `async def` routes run on one event loop. `graph.invoke(...)` is a *synchronous*, possibly slow (multiple LLM calls, revision rounds) function — calling it directly inside an `async def` route blocks that entire event loop until it finishes, meaning a second request can't even start being handled until the first one's graph run completes. Not a bug you'll see with one request at a time in testing — exactly the kind of thing that only shows up under real concurrent traffic.

The extra pieces:

- `try/except Exception` wrapped around the `graph.invoke(...)` call, logging the real exception (with `logging`, not `print`) and raising a FastAPI `HTTPException(status_code=500, detail="Task failed – please try again")` instead of letting the raw exception propagate.
- `from starlette.concurrency import run_in_threadpool` — `await run_in_threadpool(graph.invoke, initial_state)` runs the blocking call in a worker thread instead of on the event loop, so other requests can still be handled while one graph run is in progress.
- Think about what a request ID or timestamp in that log line buys you — you won't fully use it until Step 2, but the habit starts here.

Sketch the `try/except` and the `run_in_threadpool` call yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both assume every request succeeds and that only one request happens at a time. Advanced handles the two ways that assumption breaks in real use — a failure the client shouldn't see the internals of, and concurrency the single-threaded event loop can't absorb on its own.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
schemas.py:
    TaskRequest: topic (text)
    TaskResponse: final_report (text), approved (yes/no)

routes.py:
    POST /run-task(request):
        result = run the graph on request.topic
        return TaskResponse with the result

    GET /health:
        return {"status": "ok"}

main.py:
    make the FastAPI app, include the routes

test: run `uvicorn api.main:app --reload`, then either open /docs in a
browser and try it, or `curl -X POST localhost:8000/run-task -d '{"topic": "elect`
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
api/schemas.py:
class TaskRequest(BaseModel):
    topic: str

class TaskResponse(BaseModel):
    final_report: str
    approved: bool

api/routes.py:
router = APIRouter()

@router.post("/run-task", response_model=TaskResponse)
def run_task(request: TaskRequest) -> TaskResponse:
    result = graph.invoke(make_initial_state(request.topic))
    return TaskResponse(
        final_report=result["final_report"],
        approved=result["review_verdict"].approved,
    )

@router.get("/health")
def health() -> dict:
    return {"status": "ok"}

api/main.py:
app = FastAPI(title="ContentForge API")
app.include_router(router)
```

Test it by actually running `uvicorn api.main:app --reload` and hitting `/docs`, `curl -ing /run-task` with a real topic, and confirming a request missing `topic` gets a `422` automatically, with no code of yours handling that case. Write it yourself, then compare against the [Solution](#).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's almost the hardened route — fill in the missing piece yourself:

```
import logging
from fastapi import APIRouter, HTTPException
from starlette.concurrency import run_in_threadpool
from api.schemas import TaskRequest, TaskResponse
from graph import graph
from state import make_initial_state

logger = logging.getLogger("contentforge.api")
router = APIRouter()

@router.post("/run-task", response_model=TaskResponse)
async def run_task(request: TaskRequest) -> TaskResponse:
    initial_state = make_initial_state(request.topic)
    try:
        # your turn: call graph.invoke via run_in_threadpool instead of
        # calling it directly, so this doesn't block the event loop
        result = ...
    except Exception:
        logger.exception("run-task failed for topic=%s", request.topic)
        raise HTTPException(status_code=500, detail="Task failed – please try again")

    return TaskResponse(
        final_report=result["final_report"],
        approved=result["review_verdict"].approved,
    )
```

Fill in the `run_in_threadpool` call, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same one route at 3 completeness levels — Basic and Intermediate both let a failure or a slow request affect every other caller. Advanced isolates both: a failure returns a safe, generic message while the real detail goes to your logs, and the blocking graph call runs off the event loop so one slow request doesn't stall every other request behind it.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)